Week 20

GRPO, RLOO and RLVR: critic-free RL for language models

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
2.1 Why PPO became the wrong tool . . . 5
2.2 Setup: token-level MDP for a language model . . . 5
2.3 RLOO — variance reduction by leave-one-out . . . 5
2.4 GRPO — group-relative normalised advantage . . . 6
2.5 ReMax and VinePPO in one paragraph each . . . 7
2.6 RLVR — reinforcement learning with verifiable rewards . . . 8
2.7 Process rewards vs outcome rewards . . . 8
2.8 The KL anchor: what it does and when to drop it . . . 9
2.9 Empirical variance scaling . . . 9

3 Algorithms 11

4 Implementation walkthrough 13
4.1 The verifier . . . 13
4.2 The advantage estimators . . . 13
4.3 The KL-to-reference penalty . . . 13
4.4 The clipped surrogate . . . 14
4.5 Verifier design checklist . . . 14
4.6 TRL recipe for scaling up . . . 15

5 Empirical results 16
5.1 What changes if we drop SFT? . . . 16

6 Exercises 19

7 Further reading 20

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Derive the REINFORCE-Leave-One-Out (RLOO) estimator from first principles and prove it is unbiased.
2. Derive the GRPO group-relative advantage and explain why it is a control-variate variance-reduction technique.
3. State the two debiasing fixes in Dr. GRPO and the four upgrades in DAPO.
4. Describe RLVR — reinforcement learning with verifiable rewards — and design a verifier for math, code, and tool use.
5. Contrast process-reward models (PRMs) with outcome-reward models (ORMs), and cite the four leading PRM datasets.
6. Implement a working GRPO loop on a verifiable-reward task from scratch and show held-out accuracy climbing during training.
7. Articulate the central safety failure modes of RLVR (spec gaming the verifier, format hacking, distributional narrowing).

1 Motivation and intuition

PPO is the workhorse of RLHF, but it carries an awkward passenger: the *critic*. A separate critic with a 7B-parameter backbone can add roughly another 7B parameters; a small value head sharing the policy backbone does not. The memory cost depends on this architecture and the training setup. Token-level advantages require value estimates trained on *the policy's own distribution*, which drifts every step.

Two 2024 papers study updates without a learned critic: Ahmadian et al., Back to Basics, arXiv:2402.14740 (RLOO) and Shao et al., DeepSeekMath, arXiv:2402.03300 (GRPO). Both observed that completions in LLM RL are typically scored at the *sequence* level ("did the final answer match?"), so the credit-assignment problem PPO's GAE was solving is largely a fiction. Replace the critic with a *group baseline*: sample G completions per prompt, use their mean as the variance reduction, and you keep almost all of PPO's stability at half the memory.

Shao et al. (2024) discuss the cost of a separate critic.

Intuition. Think of grading G students on the same exam. Their mean score is a clean, biased-free estimate of "how hard the exam was", so subtracting it from each student's score isolates "how well *this* student did relative to the class". That isolated signal is your gradient. No instructor needs to predict a student's score in advance (the value head); the class itself provides the baseline.

The story then forks. **RLOO** keeps the sequence-level treatment clean: one sample, one scalar advantage, one importance-weighted gradient. **GRPO** broadcasts the advantage to every token and adds a PPO-style clip plus KL-to-reference — it is closer to "critic-free PPO". We will derive both, and a 2025 debiasing patch (Dr. GRPO, Liu et al.) that fixes a length bias hiding inside GRPO's per-token normalisation.

Finally, the real point of this week is what these algorithms run *on*: **RLVR**, reinforcement learning with verifiable rewards. Math has ground truth. Code has a unit-test executor. SAT-style logic has a checker. Replace the reward model with a rule, and you get the recipe that produced DeepSeek-R1, Kimi K1.5, Tülu-3 and Open-Reasoner-Zero in late 2024 and early 2025.

2 Mathematical formulation

2.1 Why PPO became the wrong tool

PPO was designed for continuous-control RL with three properties that language modelling does *not* share:

1. **Dense per-step reward.** A walking robot earns a few units of reward every control step; an LLM earns one reward at the end of a 512-token generation. PPO's per-token advantage via GAE was solving a problem (smooth credit-assignment over hundreds of controls) that simply does not exist in the LLM setting.
2. **Stationary state distribution.** A continuous-control environment hands you a stable state distribution to fit a value function against; an LLM generates its own context, so the value head is trying to fit a moving target every step.
3. **Cheap rollouts.** Mujoco simulates a robot in microseconds. Generating a 512-token completion from a 7B model costs hundreds of milliseconds; the rollout cost vastly dominates the gradient cost, which inverts PPO's design trade-off between sample efficiency and gradient accuracy.

The critic-free family replaces a learned baseline with one estimated from other completions for the same prompt. With independent on-policy samples, a leave-one-out baseline preserves the score-function gradient in expectation. A mean including the current sample rescales that gradient; dividing by a random group standard deviation changes prompt weighting as well.

2.2 Setup: token-level MDP for a language model

A prompt $x = x_{1:m}$ is fixed; the policy π_θ generates a completion $y = y_{1:T}$ token by token, $y_t \sim \pi_\theta(\cdot \mid x, y_{<t})$. A scalar reward $r(x, y) \in \mathbb{R}$ is delivered at the end. The RL objective is

$$J(\theta) \;=\; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big].$$

The policy-gradient theorem on this finite-horizon, terminal-reward MDP yields

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{x,y\sim\pi_\theta}\Big[\, r(x,y) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \,\Big]. \tag{2.1}$$

Definition 2.2.1 (Score function)**.** $s_\theta(y_t; x, y_{<t}) \triangleq \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})$. The factor $\sum_t s_\theta$ is the *sequence-level score*.

The only knob in this entire week is how we estimate $r(x, y) - b(\cdot)$ for some baseline b that does not depend on the sampled y.

2.3 RLOO — variance reduction by leave-one-out

Definition 2.3.1 (RLOO advantage)**.** Sample G i.i.d. completions $y^{(1)}, \ldots, y^{(G)}$ from $\pi_\theta(\cdot \mid x)$ and score them, $r_i = r(x, y^{(i)})$. The RLOO advantage is

$$\hat{A}_i^{\mathrm{RLOO}} \;=\; r_i \;-\; \frac{1}{G-1}\sum_{j\neq i} r_j \;=\; \frac{G}{G-1}(r_i - \bar{r}), \qquad \bar{r} = \tfrac{1}{G}\sum_j r_j.$$

Theorem 2.3.2 (Unbiasedness of RLOO). *For $G \geq 2$ independent on-policy samples from π_θ, a parameter-independent reward, and integrable score-weighted rewards, $\mathbb{E}[\hat{A}_i^{RLOO}\, s_\theta(y^{(i)})] = \nabla_\theta J(\theta)$.*

Proof. The leave-one-out baseline $b_i \triangleq \frac{1}{G-1}\sum_{j\neq i} r_j$ is a deterministic function of $\{y^{(j)}\}_{j\neq i}$ and is therefore independent of $y^{(i)}$ under i.i.d. sampling. Hence $\mathbb{E}[b_i\, s_\theta(y^{(i)})] = \mathbb{E}[b_i]\cdot\mathbb{E}[s_\theta(y^{(i)})]$, and the expected score is zero (the score-function identity: $\mathbb{E}_{y\sim\pi}[\nabla \log \pi(y)] = \nabla \sum_y \pi(y) = 0$). The first term gives the policy gradient (2.1). □ □

Theorem 2.3.3 (Variance-optimal constant baseline). *Let $S(y) = \nabla_\theta \log \pi_\theta(y \mid x)$ denote the sequence score. For a constant baseline b, finite second moments, and $\mathbb{E}[\|S\|^2] > 0$, the baseline minimising the trace of the gradient covariance is*

$$b^\star = \frac{\mathbb{E}[r\|S\|^2]}{\mathbb{E}[\|S\|^2]}.$$

This equals $\mathbb{E}[r]$ only when the reward and squared score norm have zero covariance. A random leave-one-out estimate also introduces baseline estimation noise, so it is not guaranteed to beat no-baseline REINFORCE.

Proof. The gradient mean $g = \mathbb{E}[(r-b)S] = \mathbb{E}[rS]$ does not depend on b. Thus the covariance trace is $\mathbb{E}[(r-b)^2\|S\|^2] - \|g\|^2$. Differentiate with respect to b, set the result to zero, and solve. □

Common pitfall

Variance of reward coefficients alone is not variance of policy gradients. Multiplying by different score vectors changes both magnitude and direction. Even under reward–squared-score independence, an individual RLOO term has $\mathbb{E}[(r-b_i)^2] = \mathrm{Var}(r)G/(G-1)$, which need not be smaller than $\mathbb{E}[r^2]$. The covariance between terms also matters when averaging a group.

2.4 GRPO — group-relative normalised advantage

Definition 2.4.1 (GRPO advantage). Given the same group $\{r_1, \ldots, r_G\}$,

$$\hat{A}_i^{\mathrm{GRPO}} = \frac{r_i - \bar{r}}{\sigma_r + \varepsilon}, \qquad \sigma_r^2 = \tfrac{1}{G}\sum_j (r_j - \bar{r})^2.$$

Plug $\hat{A}_i$ into a PPO-clipped surrogate, broadcast to every token, and add a KL-to-reference:

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = -\mathbb{E}_{x,\{y^{(i)}\}}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y^{(i)}|}\sum_{t=1}^{|y^{(i)}|}\Big(\min(\rho_{i,t}\hat{A}_i,\ \mathrm{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i) - \beta\,\mathrm{KL}_{i,t}[\pi_\theta \,\|\, \pi_{\mathrm{ref}}]\Big)\Bigg], \tag{2.2}$$

The σ_r divisor normalises the advantage to roughly unit variance per group — handy for a single global learning rate across very different prompts.

where $\rho_{i,t} = \pi_\theta(y_t^{(i)} \mid \cdot)/\pi_{\theta_{\mathrm{old}}}(y_t^{(i)} \mid \cdot)$ is the per-token importance ratio.

Theorem 2.4.2 (GRPO advantage is mean-zero by construction). *$\sum_i \hat{A}_i^{GRPO} = 0$ for every group. This is a coefficient identity, not a guarantee about group probability: score vectors, token lengths, clipping, and shared parameters also affect the update.*

Proof. $\sum_i (r_i - \bar{r}) = 0$ by definition of $\bar{r}$, and dividing by $\sigma_r + \varepsilon$ preserves zero sum. □ □

Dr. GRPO — removing the length and difficulty biases

Liu et al., 2025, arXiv:2503.20783 identify two biases hiding in the GRPO loss (2.2):

1. **Length bias.** Multiplying by $1/|y^{(i)}|$ gives each token in a longer response a smaller coefficient. This changes relative sequence weighting; its effect on length depends on reward sign and the correlation between length and correctness.
2. **Difficulty bias.** Dividing by σ_r *amplifies* gradients on prompts where the group has small intra-variance (near-uniform but mixed rewards) relative to higher-variance groups. Exactly uniform rewards give zero centred advantages, so their reward gradient is zero even when the standard deviation is zero.

The Dr. GRPO patch is two lines: `(i)` drop the $1/|y|$ divisor (sum the token losses, do not average them); `(ii)` use the un-normalised advantage $r_i - \bar{r}$ (or normalise across the dataset, not the group). Empirically these two patches alone produce a noticeable jump in MATH accuracy and *shorter*, not longer, chains of thought.

Common pitfall

The Dr. GRPO fixes are also a good cautionary tale: the original GRPO loss *looks* like the natural per-token cross-entropy formulation a deep learning practitioner would write. The bug was invisible until the team ran a careful length/difficulty diagnostic. Always plot the distribution of completion lengths during RL training.

DAPO — four upgrades for production-scale GRPO

Yu et al., DAPO, arXiv:2503.14476 stack four further tricks on top of GRPO:

- *Clip-Higher*: asymmetric clip $\varepsilon_{\text{low}} \neq \varepsilon_{\text{high}}$, usually 0.2 versus 0.28. Lets low-probability exploration tokens get probability mass without being capped on the upside.
- *Dynamic Sampling*: discard prompts whose group accuracy is 0 or 1 (they yield zero advantage and waste gradient capacity).
- *Token-level loss*: aggregate over valid tokens across the batch. This differs from Dr. GRPO's fixed normalisation; both remove per-response length denominators.
- *Overlong reward shaping*: a soft penalty on completions that hit the truncation budget, preventing catastrophic gradient spikes when the policy starts "thinking out loud forever".

2.5 ReMax and VinePPO in one paragraph each

ReMax (Li et al., arXiv:2310.10505) replaces the LOO/group baseline with the *greedy* reward r^{greedy}: deterministic decode once, use that as a constant baseline. One extra forward, no value net, and very low variance whenever the greedy decode is informative — which is most of the time for instruction-following RLHF.

VinePPO (Kazemnejad et al., arXiv:2410.01679) goes the other way: keep PPO's per-token credit assignment, but estimate $V(s)$ on the fly with K short Monte-Carlo rollouts ("vines") branched from every prefix. Eliminates the learned critic, gives per-step process rewards, and converges with 9× fewer iterations than PPO on MATH. The price is the extra rollouts.

2.6 RLVR — reinforcement learning with verifiable rewards

Definition 2.6.1 (Verifiable-reward task)**.** A task is *verifiable* when there exists a deterministic rule verify: $(x, y) \mapsto \{0, 1\}$ that returns 1 iff y contains the right answer (or runs cleanly through a code sandbox, or passes a formal proof checker).

The standard ingredients of an RLVR recipe (DeepSeek-R1, Tülu-3, Open-Reasoner-Zero):

1. A **rule reward** — exact-match on the final number, symbolic-equivalence via `sympy`, unit tests in a sandboxed executor.
2. Optional **format reward** — +0.1 if the completion contains `<think>...</think>` tags or a `####`-prefixed final answer. Watch out: this is the most common reward hack surface in RLVR.
3. A **KL anchor** to the SFT initialisation, often with $\beta = 0.01$–0.04, sometimes dropped entirely (Open-Reasoner-Zero argues KL is unnecessary on verifiable tasks; R1-Zero agrees).
4. A critic-free policy gradient — GRPO or RLOO — with $G = 8$–16.

Verifier design

For math, the canonical pipeline is: `(1)` strip the completion to the last line; `(2)` extract via regex `####\s*(-?[\d./]+)`; `(3)` pass the candidate and ground truth through `sympy.simplify` and accept iff their difference is zero. For code, `(1)` parse out a ```` ```python ````-fenced block; `(2)` run it against unit tests inside a `seccomp`/`firejail` sandbox with a 5 s wall-clock; `(3)` reward 1 iff all tests pass. For tool use, the verifier inspects the sequence of tool calls and matches against an oracle trace; partial credit is risky and often hacked.

2.7 Process rewards vs outcome rewards

Definition 2.7.1 (ORM)**.** An *outcome reward model* returns a scalar on the full (x, y) pair and ignores intermediate steps. Exact-match is an ORM.

Definition 2.7.2 (PRM)**.** A *process reward model* returns a scalar per step $r(x, y_{1:t})$ for every t. The trajectory reward is then aggregated — typically as the minimum, the mean, or a product of step probabilities.

Four PRM datasets dominate the literature:

- PRM800K (Lightman et al., 2023, arXiv:2305.20050) — 800K human step labels on MATH. First showed PRMs beat ORMs at best-of-N re-ranking on MATH.
- Math-Shepherd (Wang et al., 2024, arXiv:2312.08935) — automatic step labels via Monte-Carlo rollouts; a step is "good" iff continuing from it disproportionately reaches the correct final answer.
- OmegaPRM (Luo et al., 2024, arXiv:2406.06592) — MCTS-based automatic PRM data generation. Drove Gemini's math gains.
- GenRM (Zhang et al., 2024, arXiv:2410.12832) — reward is the probability of the token "yes" under a judge LLM prompted with a rubric. Rubric-RM / OpenRubrics (arXiv:2510.07743, 2025) extends this with synthetic rubrics.

Intuition. PRMs shine when the search space is large and pruning early matters — MCTS, beam search, best-of-N over long chains. ORMs win on RLVR with cheap exact-match: the marginal accuracy gain of step labels rarely justifies the labelling cost and they enlarge the reward-hacking surface (every step becomes a place to game).

2.8 The KL anchor: what it does and when to drop it

The KL-to-reference term $\beta\,\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$ serves three purposes that are worth distinguishing:

1. **Trust region.** It prevents the policy from drifting so far in one step that the importance ratio ρ explodes and the clipped surrogate degenerates. PPO's clip does some of this job already; the KL term is the safety net.
2. **Prior preservation.** It anchors the policy near a known-good reference (the SFT model). This is essential in preference-RLHF where the reward model is itself fit only locally; if the policy wanders into reward-model out-of-distribution territory, you have no guarantee the reward signal is meaningful.
3. **Skill retention.** It implicitly fights catastrophic forgetting of non-rewarded skills. The SFT model knew how to do creative writing and dialogue; pure RLVR shrinks that distribution.

For *verifiable* tasks (purposes 1 and 3 only), Open-Reasoner-Zero and R1-Zero report that the KL term is essentially dispensable when the base model is strong enough: the reward signal is grounded in external truth, so there is no out-of-distribution risk on the reward side, and the clip alone provides the trust region. For *preference* RLHF (purpose 2), removing the KL anchor reliably explodes the policy into reward-model artifacts within tens of steps.

For samples from p with common positive support for p and q, define

$$\hat{k}^3 = e^{\log q - \log p} - (\log q - \log p) - 1.$$

Since $\mathbb{E}_p[q/p] = 1$, $\mathbb{E}_p[\hat{k}^3] = \mathrm{KL}(p\|q)$. The sample is non-negative because $e^u \geq 1 + u$. Its variance can be lower near $p = q$; this is not a universal variance bound. If samples instead come from an older policy, the same unweighted average is not generally an unbiased estimate of the current-policy KL.

Common pitfall

The signed estimator $\log p - \log q$ is valid in expectation under p; a negative sample is not a bug. Also distinguish an unbiased scalar KL estimate from its gradient: differentiating an estimate while treating samples as fixed omits the derivative of the sampling distribution. Choose reward shaping, an exact categorical KL, or a documented surrogate with the appropriate sampling/importance-weight convention.

2.9 Empirical variance scaling

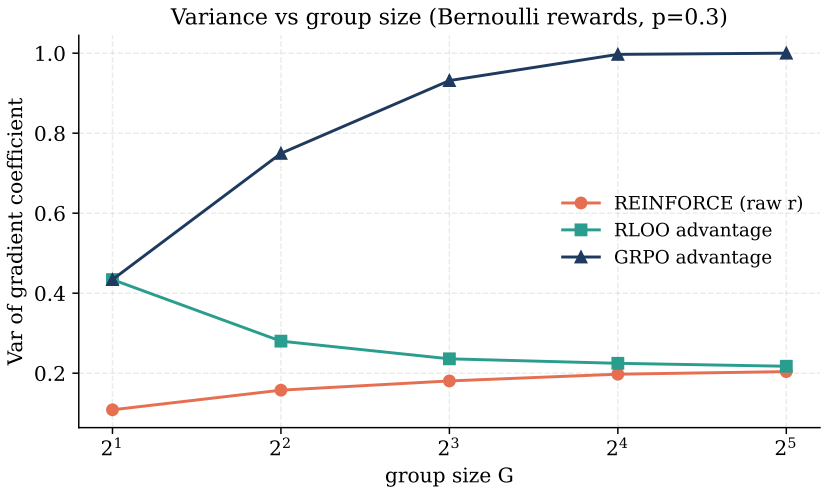


Figure 2.1: Variance of the gradient coefficient under three estimators as group size grows. Bernoulli rewards with $p = 0.3$. RLOO and GRPO are both group-baseline control variates; GRPO additionally divides by group σ_r, which on this Bernoulli case happens to inflate variance for mid-sized groups before settling. The plot motivates the Dr. GRPO debias.

3 Algorithms

Algorithm: GRPO step (DeepSeekMath, 2024)

Input. prompt x, group size G, current policy π_θ, reference policy π_{ref}, KL coefficient β, clip ε.
1. Sample G completions $\{y^{(i)}\}$ from $\pi_{\theta_{\text{old}}}(\cdot \mid x)$.
2. Compute rewards $r_i = r(x, y^{(i)})$ via the verifier.
3. Form group baseline $\bar{r}$ and (optionally) σ_r; compute $\hat{A}_i = (r_i - \bar{r})/(\sigma_r + \varepsilon)$.
4. For every token in every $y^{(i)}$, compute the importance ratio $\rho_{i,t}$ and the unbiased KL estimate $\hat{k}^3_{i,t} = e^{\log \pi_{\text{ref}} - \log \pi_\theta} - (\log \pi_{\text{ref}} - \log \pi_\theta) - 1$.
5. Form the clipped surrogate (2.2), take an Adam step, clip gradient norm.

Algorithm: RLOO step (Ahmadian et al., 2024)

Input. prompt x, group size G, current policy π_θ.
1. Sample G completions $\{y^{(i)}\}$ from $\pi_\theta(\cdot \mid x)$, caching their on-policy log-probs.
2. Compute rewards $r_i = r(x, y^{(i)})$ via the verifier.
3. Form per-sample LOO baselines $b_i = \frac{1}{G-1}\sum_{j \neq i} r_j$ and advantages $\hat{A}_i = r_i - b_i$.
4. Treat each completion as a single action with sequence-level log-probability $\log \pi_\theta(y^{(i)} \mid x) = \sum_t \log \pi_\theta(y^{(i)}_t \mid \cdot)$.
5. Loss $\mathcal{L} = -\frac{1}{G}\sum_i \hat{A}_i \log \pi_\theta(y^{(i)} \mid x)$.
6. Optional: PPO clip on the sequence-level importance ratio if you multi-step the same batch.

Algorithm: Dr.GRPO step (Liu et al., 2025)

Input. as for GRPO, plus the debiased flag.
1–2. Identical to GRPO.
3. Form $\hat{A}_i = r_i - \bar{r}$ *(no division by σ_r).*
4–5. Identical to GRPO clipped surrogate.
6. Token loss is *summed* over tokens, not averaged (removes the $1/|y|$ length bias).
7. Mean over sequences in the batch (yes), Adam step, clip grad norm.

Worked numerical example

A single prompt $x =$ "$3 + 4 =$". Sample $G = 4$ completions: $y^{(1)} =$ "7<eos>" ($r_1 = 1$), $y^{(2)} =$ "5<eos>" ($r_2 = 0$), $y^{(3)} =$ "7<eos>" ($r_3 = 1$), $y^{(4)} =$ "==" ($r_4 = 0$, no number found).

$\bar{r} = 0.5$, $\sigma_r = 0.5$. GRPO advantages $\hat{A} = (1, -1, 1, -1)$ (unit normalised). RLOO advantages $\hat{A}^{\text{RLOO}} = \frac{4}{3}(0.5, -0.5, 0.5, -0.5) = (\frac{2}{3}, -\frac{2}{3}, \frac{2}{3}, -\frac{2}{3})$. Dr. GRPO advantages $\hat{A}^{\text{Dr}} = (0.5, -0.5, 0.5, -0.5)$.

Notice: all three estimators give exactly the same *sign* on every sample. The differences are scale (which the learning rate absorbs) and the implicit weighting between prompts with high vs low intra-variance (which Dr. GRPO targets).

Complexity

Per GRPO step on a sequence of length T, group size G, prompts P: $\mathcal{O}(PG \cdot T \cdot d^2)$ forward, identical backward; the verifier adds $\mathcal{O}(PG)$ regex evaluations (negligible) or sandboxed execution (dominant if you use code rewards).

4 Implementation walkthrough

The mini-project lives under `project/`. Three files matter: `env.py` (toy arithmetic + verifier), `agent.py` (a tiny $\sim$ 30k-parameter char-level Transformer), and `train.py` (the GRPO loop with RLOO and Dr. GRPO switches). On a single CPU thread, the full train converges in $\sim 15\,\text{s}$.

4.1 The verifier

```python
_NUM_RE = re.compile(r"-?\d+")

def extract_final_number(text):
    matches = _NUM_RE.findall(text)
    if not matches:
        return None
    try:
        return int(matches[-1])
    except ValueError:
        return None

def exact_match_reward(completion, target):
    pred = extract_final_number(completion)
    return 1.0 if pred is not None and pred == target else 0.0
```

That is the entire RLVR reward for this week. Note the explicit guard against "no number found" — a model that emits only `====` would otherwise compare `None == target` and crash, or worse, silently match `None == None` on some Python paths.

4.2 The advantage estimators

```python
def grpo_advantages(rewards, eps=1e-6, debias=False):
    mean = rewards.mean(dim=-1, keepdim=True)
    if debias:                              # Dr.GRPO
        return rewards - mean
    std = rewards.std(dim=-1, keepdim=True, unbiased=False)
    return (rewards - mean) / (std + eps)

def rloo_advantages(rewards):
    G = rewards.shape[-1]
    mean = rewards.mean(dim=-1, keepdim=True)
    return (G / (G - 1)) * (rewards - mean)
```

4.3 The KL-to-reference penalty

We use Schulman's k3 unbiased estimator (lower variance than the naive $\log p - \log q$ form, always non-negative):

$$\hat{k}^3 \;=\; \exp(\log q - \log p) - (\log q - \log p) - 1, \qquad \mathbb{E}_{x\sim p}[\hat{k}^3] = \mathrm{KL}(p \,\|\, q).$$

```
log_ratio = ref_logps - new_logps
kl_tok = (torch.exp(log_ratio) - log_ratio - 1.0) * mask
```

4.4 The clipped surrogate

```
ratio = torch.exp(new_logps - sample_logps.detach())
surr1 = ratio * adv.unsqueeze(-1)
surr2 = torch.clamp(ratio, 1-eps, 1+eps) * adv.unsqueeze(-1)
pg_loss_tok = -torch.min(surr1, surr2) * mask
```

4.5 Verifier design checklist

A robust verifier for a single math problem on GSM8K looks like:

```
import re, sympy as sp

GSM8K_FINAL_RE = re.compile(r"####\s*([-+]?\d+(?:\.\d+)?)")
ANY_NUMBER_RE  = re.compile(r"[-+]?\d+(?:\.\d+)?")

def gsm8k_verify(completion: str, target: float) -> float:
    # 1. Preferred: the model emitted the canonical #### marker.
    m = GSM8K_FINAL_RE.search(completion)
    if m is None:
        # 2. Fallback: the last number in the completion.
        nums = ANY_NUMBER_RE.findall(completion)
        if not nums:
            return 0.0
        pred_s = nums[-1]
    else:
        pred_s = m.group(1)
    try:
        pred = sp.Rational(pred_s)
        return 1.0 if sp.simplify(pred - sp.Rational(target)) == 0 else 0.0
    except Exception:
        return 0.0
```

The three safety considerations encoded above:

- Prefer the canonical marker over the fallback regex — the marker is harder to game accidentally.
- Use `sympy.Rational` for equivalence; string equality fails on `"2.0"` vs `"2"` and on `"1/2"` vs `"0.5"`.
- Wrap the whole thing in `try / except` — a verifier that crashes on malformed input is a denial-of-service vector during RL.

Common pitfall

A common verifier hack: when the regex matches and `sympy` fails, return 1 for charity. This grants the policy free reward whenever it emits unparseable nonsense after the marker. Always return 0 on verifier failure, never the benefit of the doubt.

4.6 TRL recipe for scaling up

The same loop on Qwen2.5-0.5B-Instruct + LoRA on a 24 GB GPU:

```
from trl import GRPOTrainer, GRPOConfig
from peft import LoraConfig

cfg = GRPOConfig(
    output_dir="runs/grpo-qwen",
    num_generations=8,
    max_completion_length=512,
    temperature=0.9,
    learning_rate=1e-6,
    beta=0.04,              # KL coefficient
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
)
peft = LoraConfig(r=32, lora_alpha=64, target_modules="all-linear")
trainer = GRPOTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    reward_funcs=[exact_match_reward],
    args=cfg,
    train_dataset=gsm8k_train,
    peft_config=peft,
)
trainer.train()
```

5 Empirical results

5.1 What changes if we drop SFT?

If we set `sft_warmup_steps=0` the GRPO curve sits flat at zero: the verifier never fires, the group baseline is always zero, the advantage is identically zero, and the gradient is zero. This is the standard *cold-start* pathology of RLVR. DeepSeek-R1-Zero famously bypassed it on a large base model; for tiny models, a short SFT bootstrap is the canonical fix.

Safety angle

Failure modes specific to RLVR.

1. *Verifier spec-gaming.* A code-verifier checking only that the unit tests pass invites `import sys; sys.exit(0)`. Defence: require the program to compute the result; sandbox; run secondary verifiers that re-execute the test harness.
2. *Format reward hacking.* If you reward +0.1 for emitting `####`, the model will emit `####` at the start of every line. Defence: only reward the format pattern when it is followed by a correctly parsed answer.
3. *Brittle math regex.* `r"\d+"` matches the year "2024" inside an explanation. Defence: parse the final line, not the whole completion; use `sympy` equivalence rather than string equality.
4. *Distributional narrowing.* Pure RLVR shrinks the policy onto the verifier-rewarded distribution and forgets unrelated skills. TÜLU-3 mixes RLVR with a small DPO stream; many recipes keep a $\beta \geq 0.01$ KL anchor to the SFT model.
5. *Length bias.* The vanilla GRPO loss systematically favours shorter completions because of its $1/|y|$ normaliser; Dr. GRPO and DAPO drop that divisor, and Kimi K1.5 adds an explicit length penalty.

The safety analogue to *Theorem 2.3.2* is sobering: the baseline subtraction is correct in expectation, but the empirical average of LOO advantages over the policy's own samples can drift by many standard deviations when the verifier rate is low — exactly the "rare-positive" regime that triggers reward-hacking incentives. See week 21 for the Gao 2022 over-optimisation scaling laws.

Matching application

Process rewards as interview-stage signals. Insurance-broker matching (synthetic insurance) and candidate-job matching (synthetic ATS) both involve a sequence of partial outcomes before a terminal one: in ATS, *shortlist* → *interview-1* → *interview-2* → *offer* → *hired*. A pure ORM reward ("did they get hired?") is sparse and slow. A PRM aggregator (the `exercise_7` solution in this week) lets the policy learn from intermediate signals — a candidate cleared interview-2 is informative even if they ultimately declined the offer.
The matching analogue of Math-Shepherd's automatic PRM is: bootstrap the step labels by Monte-Carlo from historical outcomes. "This shortlist profile predicts 70% hire-rate"

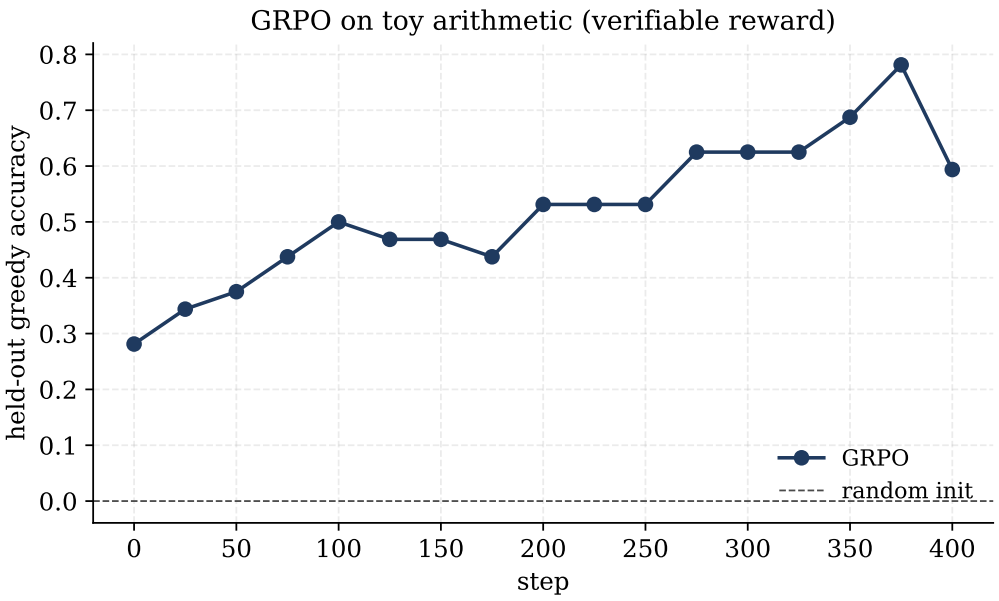


Figure 5.1: Held-out greedy accuracy on the toy arithmetic task, evaluated every 25 GRPO steps. The post-SFT initialisation lands near 0.45; 500 GRPO steps push it to ~ 0.65.

becomes the step reward, the way "this proof step continues to a correct final proof in 70% of rollouts" becomes the step label in PRM800K.

Practical caveat. PRMs expand the gameable surface — a candidate-ranking policy can learn to satisfy the "shortlist-stage signal" (resume keywords) without improving downstream hire rate. The pattern is identical to RLVR format hacking; see week 21 for the mitigation toolkit.

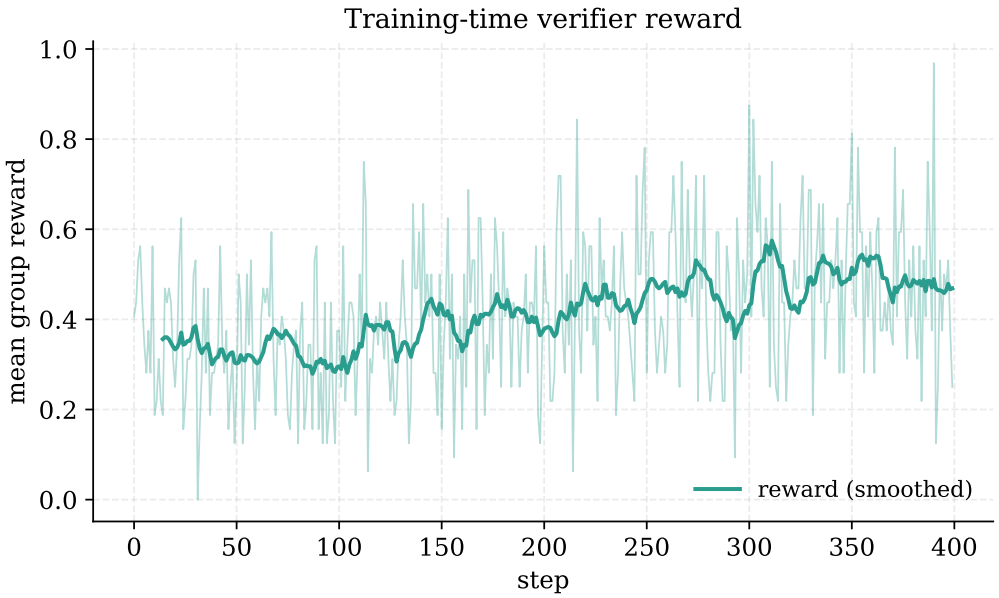


Figure 5.2: Training-time mean group reward (raw and 15-step smoothed). Reward is the binary verifier output. The variance is consistent with the $p(1-p)$ Bernoulli bound at $p \sim 0.65$.

6 Exercises

See `workbook.ipynb` for fully spelled-out cells with hints; the summary list:

- ★ Implement `extract_final_number` matching the GSM8K `####` convention.
- ★★ Implement the RLOO advantage from scratch and prove it is mean-zero.
- ★★ Implement GRPO with and without standard-deviation normalisation (Dr. GRPO) and reproduce the variance plot (Figure 2.1).
- ★★ Empirically confirm Theorem 2.3.3 on Bernoulli rewards.
- ★★ Implement the unbiased k3 KL estimator and show it is non-negative across 10^4 Monte-Carlo trials.
- ★★ Code the PPO-clipped surrogate on a single token and sketch the loss landscape as a function of the importance ratio.
- ★★ *Matching-themed.* Implement three PRM aggregators (`min`, `mean`, `prod`) and discuss which is appropriate for the ATS interview-stage analogue.
- ★★★ *Open-ended.* Design a shaped reward with a format bonus, train with GRPO, and quantify the format-hacking failure mode by plotting "rate of any-number-emitted" vs "rate of correct-number-emitted".

7 Further reading

See `extended_reading.pdf` for the curated five papers (DeepSeekMath, RLOO, Dr. GRPO, DeepSeek-R1, PRM800K). Section 3 of `research/llm_rl_sota.md` carries the broader 2024–2026 dossier; the verifier-design checklist in section 4 of that file is what we operationalised above.